

Flow persona: A QoS Flow Rule Scheme based on Deep Learning in SDN-IoT

Hao She^{a,b,c}, Lixing Yan^{a,b,c}, Xin An^{a,b,c}, Chuanfeng Mao^{a,b,c}, Yongan Guo^{a,b,c,*}

^a College of Telecommunications & Information Engineering, Nanjing University of Posts and Telecommunications, Nanjing, 210003, China

^b Jiangsu Key Laboratory of Intelligent Information Processing and Communication Technology, Nanjing University of Posts and Telecommunications, Nanjing, 210003, China

^c Edge Intelligence Research Institute, Nanjing University of Posts and Telecommunications, Nanjing, 210003, China

ARTICLE INFO

Keywords:

SDN-IoT
Flow Persona
ARIMA
APSO
QoS flow classification

ABSTRACT

With the proliferation of Internet of Things (IoT) devices and increasing network flow, traditional network architectures struggle to manage complex flow and meet evolving Quality of Service (QoS) requirements. These architectures lack flexibility in resource allocation and optimization, limiting their support for diverse IoT applications. To address these issues, we propose a QoS Flow Rule Scheme based on Deep Learning in Software Defined Networking-IoT (SDN-IoT) called Flow Persona. This scheme integrates user personas and QoS requirements, employs an ARIMA model for traffic prediction, and leverages a Convolutional Neural Network (CNN) optimized by Adaptive Particle Swarm Optimization (APSO) for flow classification. Simulation results show that flow persona improves QoS flow classification accuracy by about 4.6% over traditional and existing algorithms. It also significantly enhances precision, recall, and F-score, while improving QoS routing efficiency and reducing network delay.

1. Introduction

In recent years, with the rapid development of network technology, the network has evolved towards intelligence. In this context, user needs and business types are becoming more diverse and dynamic. For example, from traditional voice and text communications to new applications such as real-time audio and video streaming, large-scale online games, and virtual reality, higher requirements are placed on network performance, stability, and response speed. However, the definition and analysis of flows in the current network are still insufficient, making it difficult to describe complex and diverse business needs accurately. This limitation hinders effective resource allocation and path selection, thereby compromising the efficiency of service quality assurance in dynamic network environments. Especially in intelligent scenarios, the lack of in-depth understanding of dynamic flow characteristics and real-time optimization mechanisms has become an important bottleneck restricting improving network routing efficiency. The network status information required by the intelligent network needs to be implemented through the Software Defined Networking (SDN) framework.

The implementation of end-to-end Quality of Service (QoS) in traditional networks faces significant challenges. SDN, a well-established

paradigm, has become a foundational technology in modern 5G and IoT communication networks [1]. By decoupling the data plane from the control plane, SDN provides a centralized global view of the network and enables flexible programmability, which allows dynamic resource management and protocol enforcement. In practical deployments such as 5G service slicing, ultra-reliable low-latency communication (URLLC), and massive IoT scenarios, SDN plays a critical role in ensuring QoS by enabling real-time flow classification, latency-aware routing, and adaptive bandwidth allocation [2]. The data plane is responsible for forwarding packets, while the control plane maintains holistic awareness and determines appropriate QoS flow rules based on global network conditions. Consequently, optimizing data transmission quality by accounting for diverse service requirements and dynamic traffic behaviors remains a key challenge and research focus in SDN-based QoS frameworks [3].

QoS evaluates the network's ability to deliver services based on parameters like bandwidth, delay, jitter, and packet loss. Traditional QoS flow classification algorithms typically make decisions based on overall network needs, neglecting the specific requirements of individual users or streams. Additionally, their complexity often hinders network managers from promptly and accurately determining appropriate QoS policies. Unlike traditional networks, SDN controllers, with their

* Corresponding author.

E-mail addresses: 2021010112@njupt.edu.cn (H. She), yanlixing58@126.com (L. Yan), xina0310@outlook.com (X. An), 2481445289@qq.com (C. Mao), guo@njupt.edu.cn (Y. Guo).

<https://doi.org/10.1016/j.comnet.2025.111784>

Received 21 April 2025; Received in revised form 21 September 2025; Accepted 13 October 2025

Available online 17 October 2025

1389-1286/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

global network view, can monitor the network, calculate protocols, and manage data flow more effectively, ensuring better adherence to QoS requirements.

Currently, the lack of consensus on QoS flow classification leads to inaccurate analysis across different services, compromising network stability and the ability to meet growing service demands. In this context, persona becomes a powerful tool by analyzing and understanding user preferences and behaviors to create a comprehensive user persona. Persona offers valuable insights for refining QoS flow classification, addressing specific user needs, enhancing network stability, and delivering high-quality services.

The integration of AI technology has introduced smarter, more efficient methods for Internet of Things (IoT) data transmission. By using deep learning prediction models, time series data from terminal devices can be analyzed and predicted, allowing dynamic adjustments to data transmission based on the time characteristics of different flows. This enhances network resource utilization and optimizes transmission. Additionally, AI can predict data flow fluctuations by analyzing historical data and recognizing patterns, enabling network managers to proactively adjust for flow peaks and troughs. This proactive approach improves resource utilization and performance. AI also facilitates real-time decisions and optimizations during data transmission, dynamically adjusting strategies to achieve optimal QoS.

Thus, we proposed a QoS flow prediction method that combines the ARIMA model and the Levinson-Durbin algorithm, which can monitor the network status in real-time and optimize flow management. By introducing the concept of flow persona, flow classification is performed based on user behavior preferences and historical data, which improves resource allocation efficiency. Finally, the APSO optimized CNN classification model we proposed effectively improves search efficiency and avoids the local optimal problem. Our main contributions are summarized as follows.

- We introduce a strategy using the Autoregressive Integrated Moving Average (ARIMA) model statistical algorithm to collect key performance indicators via the NetFlow protocol and other methods. Differential processing is then applied to remove trend components and ensure data stability. The Levinson-Durbin algorithm refines the parameters of autoregression and moving average to accurately fit QoS flow time series data. This approach not only enhances prediction accuracy but also improves network flow management and user experience, ensuring that the proposed QoS flow rule scheme can monitor network status in real-time and converge quickly.
- By integrating QoS flow rule with user persona, flow categorization based on data characteristics and QoS requirements is enabled. This creates 'flow persona' that describe and analyze each category. Flow persona visualize network conditions, and optimize resource allocation. This approach improves network flow management, enhancing service quality and resource efficiency.
- This paper presents a classification model that utilizes Adaptive Particle Swarm Optimization (APSO) optimization Convolutional Neural Network (CNN). The proposed model improves exploration of the search space, leading to faster convergence towards the optimal solution. Moreover, it effectively addresses the issue of getting trapped in local optima by overcoming the limitations posed by inappropriate initial positions.

The structure of this paper is outlined as follows: [Section 2](#) provides a concise overview of the related work and the motivation behind our research. [Section 3](#) introduces the system model. The implementation details of our approach are described in [Section 4](#). The convergence and performance evaluation of our approach are presented in [Section 5](#). Finally, [Section 6](#) concludes the paper.

2. Background and Motivation

2.1. Persona

Persona, a marketing tool with significant strategic value, was first introduced by Alan Cooper in 1999 [4], focusing on developing user-centered profiles. It involves creating a highly realistic and representative fictional user image [5]. With the rapid advancement of technology, the application of persona has extended well beyond the design field [6],[7]. Leveraging its sharp insight into users' real and potential needs, persona enables refined behavioral feature cluster analysis using tag attributes. This approach has significantly enhanced the efficiency of matching supply and demand in public cultural services and improved the differentiation in resource integration [7]. In cutting-edge areas like e-commerce, social media, fintech, healthcare, and short video recommendations, persona has demonstrated strong adaptability and practicality, becoming a driving force for industry innovation and development [8].

However, the research and application of persona in the crucial area of network analysis and management remain limited [9],[10]. Challenges such as the insufficient accuracy of user behavior prediction and inflexible QoS adjustments highlight the need for enhanced tools. Persona, with its powerful user behavior analysis capabilities, has the potential to address these issues [11]. By deeply analyzing user behavior patterns, persona can offer precise demand forecasts and dynamic service adjustment strategies, driving network services towards greater personalization and accuracy. This, in turn, can significantly enhance network management and QoS. Therefore, exploring the potential of persona in QoS optimization and flow analysis is essential for fostering the sustainable and healthy development of the network industry.

2.2. QoS Flow

In traditional network architecture, QoS flow classification policies are widely used to optimize network performance and resource allocation by distinguishing and prioritizing critical data flow to meet the performance needs of various applications. Wang et al. [12] introduces an innovative approach that combines feature generation with a Long Short-Term Memory (LSTM) model, providing a new method for QoS flow classification. This approach formulates forwarding rules based on the characteristics of different data flow, ensuring the reliability and stability of QoS, thereby significantly enhancing user experience. Garzon et al. [13] addresses the flow distribution challenge in 5G backhaul networks by integrating precise optimization with heuristic techniques, which ensures a high flow acceptance rate and meets the deterministic QoS requirements of critical flows. Hu et al. [14] explores a flow classification method based on graph neural networks, which markedly improves QoS performance and classification accuracy by converting network packets into undirected graphs and utilizing two-layer graph convolutional networks along with three distinct aggregation strategies. Zhang et al. [15] proposes an adaptive model update mechanism that effectively reduces data transmission delays and packet loss by accurately identifying and prioritizing key data streams, thereby ensuring the QoS requirements of various applications.

In an SDN environment, the complexity of network management increases significantly, necessitating a comprehensive approach that considers routing policies, load balancing, and QoS assurance. Ibrar et al. [16] introduces a flow allocation algorithm for smart cities based on SDN and fog computing, which enhances the performance of delay-sensitive services by taking into account the reliability level of links during flow allocation. Huang et al. [1] presents a near-optimal flow control method for hybrid SDN, exploring SDN migration sequences to maximize controllable flow, and achieves QoS optimization through deep reinforcement learning algorithms. Liu et al. [17] combines SDN with a LSTM network and a proximal policy optimization algorithm to perform feature extraction, state perception, and decision-making,

generating dynamic flow scheduling policies that meet QoS objectives in service scenarios, thereby further enhancing QoS guarantee capabilities. Saha et al. [18] discusses the implementation of QoS-based flow rule aggregation in SDN networks, highlighting how the Q-Flag algorithm can significantly improve network performance and service quality, ensuring QoS reliability and stability, and providing a better user experience and service assurance for IoT applications. Although these studies have made significant strides in designing QoS flow rules, challenges remain in adapting to the rapidly changing network environment. For instance, existing schemes often lack the agility to quickly adjust to dynamic changes, potentially causing delays and misjudgments in real-world applications. This limitation makes it difficult to meet the demands for fine-grained QoS guarantees and real-time responsiveness. Additionally, network managers often struggle to fully understand the characteristics of each data stream, complicating the scheduling and management of refined QoS flows, which ultimately impacts end-user experience and satisfaction.

2.3. AI-Based Network Flow Classification

Network flow classification is crucial for network measurement and management. Emerging AI algorithms have become viable solutions for flow classification and are widely applied in data flow classification tasks. Suguna et al. [19] proposes a heterogeneous network flow classification method based on ensemble learning, called Cat-ANTC. This method uses the CatBoost algorithm to automatically classify multiple types of flow, achieving high prediction accuracy and incorporating model regularization to reduce overfitting and improve efficiency effectively. Xie et al. [20] proposes a two-stage online elephant flow prediction method using data flow mining. By mapping the trained classification and prediction decision tree models to the corresponding action pipeline of a programmable switch, the models and parameters can be updated online, enabling dynamic load balancing. Zhang et al. [21] proposes an autonomous model update mechanism for AI-based flow classifiers, focusing on scenarios where these classifiers are used to handle network flow from both known and unknown applications.

Although existing research has made progress in applying artificial intelligence to classify network flow, several limitations remain. Many methods struggle with the dynamic nature and diverse needs of heterogeneous network flow, resulting in weak generalization ability and difficulty adapting to complex network environments [22]. Additionally, traditional models rely heavily on hyperparameters and feature selection, and improper parameter settings can negatively impact classification performance [23], [24]. The lack of real-time processing and resource efficiency further limits the applicability of these methods in high-flow scenarios [25]. To address these challenges, this paper proposes an innovative approach that combines QoS flow rules and persona construction, an efficient time-series prediction strategy, and an APSO optimization CNN classification model. This approach not only enhances prediction accuracy and resource management efficiency but also optimizes user experience, providing more reliable support for high-quality services in complex network environments.

2.4. Motivation

With the rise of diverse new services in the 5G and cloud era, and the rapid development of IoT, cloud storage networks, and SDN, QoS flow classification faces new challenges. Integrating persona into QoS flow classification is expected to enhance service personalization and accuracy. Persona improves network service efficiency by analyzing user behavior and historical data, ensuring a consistently high-quality QoS experience. Despite SDN's flexibility and scalability, QoS flow classification models still face issues with accuracy and convergence speed, largely due to manual parameter adjustments and a lack of intelligent, dynamic optimization methods.

To address these issues, we propose a QoS flow rule scheme called 'flow persona', to optimize flow transport in the SDN-IoT environment. This scheme integrates persona into network flow management by analyzing and modeling user characteristics and behaviors using DL models. It classifies and schedules network flow based on user needs and preferences, offering more personalized and efficient service.

Recent advancements such as RDG-TE [26] have demonstrated the effectiveness of combining deep reinforcement learning with graph neural networks to optimize traffic engineering strategies in SDN. These approaches primarily focus on network-centric objectives, utilizing topology-aware metrics such as path reliability, latency, and congestion for routing optimization. In contrast, the FlowPersona framework proposed in this work adopts a user-centric modeling perspective by incorporating behavioral preferences, historical service patterns, and context-aware flow differentiation. Instead of optimizing routing paths directly, FlowPersona generates adaptive flow rules aligned with user-level QoS expectations, enabling personalized and service-intent-aware traffic management.

To further clarify this distinction and enhance the positioning of our method, Table 1 provides a conceptual comparison between FlowPersona and representative DRL-based flow optimization and classification methods. Key differences in modeling perspective, input features, learning mechanisms, and system-level objectives are summarized to highlight the unique contributions of FlowPersona.

The Flow Persona mechanism is designed to support differentiated QoS slicing in SDN-IoT environments, enabling the classification and management of flows according to service-specific requirements. In particular, the proposed scheme targets three primary slice types: 1) latency-sensitive slices for services such as smart healthcare and industrial control systems; 2) high-throughput slices for video surveillance and real-time media streaming; and 3) delay-tolerant slices for applications like environmental monitoring and smart agriculture. By aligning flow rules with user behavior and service context, Flow Persona enables intent-aware, slice-adaptive flow control that enhances service quality across heterogeneous IoT domains.

3. System Model

In this section, we will introduce the network model and flow persona model that we have developed. Our research is centered around leveraging flow persona to establish a QoS flow rule scheme, enabling effective QoS flow analysis and efficient classification. Please refer to Table 2 for a list of symbols utilized in this article.

3.1. Network Model

Next, we will establish specific configurations for the designed network model. We develop a model architecture for wireless networks that considers user needs, business goals, and network dynamics in SDN-IoT settings. To achieve an effective QoS flow rule scheme, we need to define and examine the network topology G , wireless transmission rate C , and quality of service Q .

- 1) Network Topology: The flow persona model discussed in this paper is designed for an SDN-IoT scenario. The network topology G we created is illustrated in Fig. 1. It features a mesh structure with numerous Segment Routing (SR) nodes deployed. This architecture provides multiple paths, ensuring network connectivity and stability even in the event of node failures. The multipath capability supports load balancing and minimizes the risk of overloading any single path, thus preserving overall network performance and efficiency. Additionally, the structure offers high flexibility and scalability, allowing network administrators to dynamically adjust routing policies based on flow demands and real-time network load, effectively meeting diverse QoS requirements.

Table 1

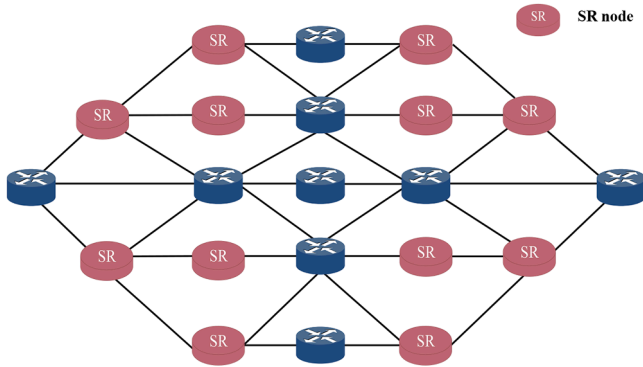
Conceptual comparison between FlowPersona and representative DRL-based flow optimization/classification methods.

Aspect	FlowPersona (This Work)	Representative DRL-Based Flow Optimization Methods
Primary Objective	QoS-aware flow classification and adaptive rule enforcement	End-to-end traffic engineering and routing optimization
Modeling Perspective	User-centric: behavior-driven and preference-aware	Network-centric: topology- and metric-driven
Input Features	Historical user behavior, service preference, QoS attributes	Link delay, reliability, congestion status, topology state
Learning Mechanism	ARIMA-based flow prediction + APSO-optimized CNN classification	DRL agents (e.g., DQN, DDPG) combined with GNN or MLP
Application Layer	Flow classification and QoS policy generation at the control plane	Routing path decision and resource allocation in SDN
Personalization Support	High: personalized rules based on user persona	Low: generalized treatment of flows

Table 2

Notations.

Notation	Description
G	Flow persona network topology diagram
C	Wireless transmission rate
Q	Referring to QoS
l	A specific flow within the graph G
$\mathcal{L}$	A collection of all flows, $l \in \mathcal{L}$
F_l	The magnitude of traffic for flow l
F	The cumulative traffic of all flows in the network, $F = \sum F_l$
$D_l(t)$	The data volume transmitted by flow l within a specific time frame, typically measured in units such as bits or bytes at time t
$T_l(t)$	The duration, expressed in seconds (s), required for flow l to transfer the data amount D_l at time t
$Th_l(t)$	The throughput of flow l at time t , $Th_l(t) = D_l(t)/T_l(t)$
$q_{de}(t)$	End-to-end delay between SR nodes at time t
$q_{th}(t)$	Throughput at time t
$q_{lo}(t)$	Package loss rate at time t
ξ_{y_i}	The difference value of the time series
y'_i	The value of the time series after difference at time i
a_i	The autoregressive coefficient of step i
$\hat{y}_{i+h}$	The predicted value of step $i + h$

**Fig. 1.** Part of the network topology.

- 2) Wireless Transmission Rate: Our QoS flow classification strategy leverages efficient wireless transmission technology to monitor flow, optimize paths in real-time, and prevent network congestion and service degradation. In both our network model and simulations, we use the 802.11ac standard to achieve high-speed wireless transmission, with a maximum transmission rate of C up to 1.3Gbps, thereby enhancing user experience. This is used solely to simulate high-bandwidth scenarios such as video streaming and does not imply applicability to all IoT contexts. In addition, other state-of-the-art wireless technologies such as 5G New Radio, Wi-Fi 6/802.11ax, and LTE-Advanced could equally serve as evaluation baselines. The Flow Persona framework remains independent of the underlying transmission technology, ensuring adaptability across heterogeneous network environments.
- 3) QoS: To address computational complexity and NP-complete issues, we establish three core indicators: delay, throughput, and packet loss rate, denoted as $Q = \{q_{de}, q_{th}, q_{lo}\}$. These indicators ensure network effectiveness and stability, enhancing QoS. While practical applications may require additional QoS factors such as availability, jitter, and bandwidth guarantee, the flow persona is an extensible model

and can be adjusted to meet varying application scenarios and user needs.

$$Q = \{q_{de}, q_{th}, q_{lo}\} \quad (1)$$

Naturally, in practical application scenarios, QoS service quality must consider additional factors, including availability, jitter, and bandwidth guarantee, among others. Nonetheless, the QoS flow rule scheme based on flow persona provides a scalable solution. It allows for flexible adjustment of rule schemes based on specific requirements, accommodating various application scenarios and user demands effectively.

In our simulation, the SDN-IoT mesh network topology consists of 50 SR nodes, forming a scalable multi-path architecture. This topology enables dynamic flow rerouting and is designed to accommodate large-scale IoT scenarios by supporting horizontal expansion with minimal reconfiguration overhead. Each SR node is equipped with basic edge computing capabilities to simulate distributed intelligence. The considered application-layer data rates are as follows: video streaming services are modeled with bitrates ranging from 2 Mbps to 5 Mbps (representing standard and high-definition streams), audio streaming services are assigned 64–128 kbps depending on codec assumptions, and ordinary data services, such as sensor telemetry or background sync, operate at 10–50 kbps. These settings reflect typical bandwidth requirements observed in practical IoT deployments.

3.2. Flow Persona Model

To illustrate the specific challenges addressed by FlowPersona, consider a simplified SDN-IoT scenario involving three representative user types:

- User1 primarily engages in ordinary data services such as browsing and background synchronization, which are delay-tolerant and bandwidth-insensitive.
- User2 frequently uses audio-related services like music streaming and voice calls, which are sensitive to delay and jitter.
- User3 is focused on video-based applications such as video calls and high-definition streaming, which require both high throughput and low latency.

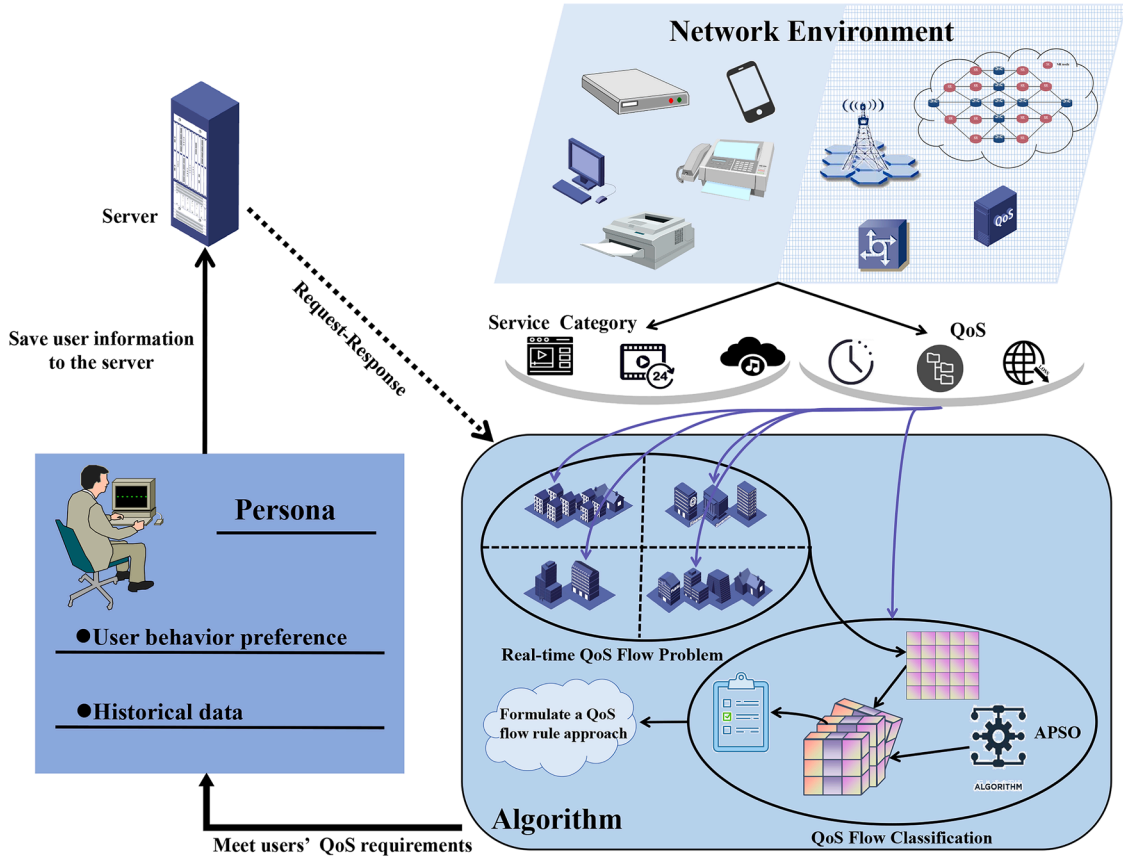


Fig. 2. Flow persona model.

In a congested network environment-due to burst traffic or limited wireless bandwidth-traditional flow classification mechanisms typically apply uniform treatment based on static QoS tags or protocol types. This can lead to inefficient resource allocation: for instance, video streams of User3 may suffer latency spikes, and audio services for User2 may experience jitter, ultimately degrading QoS.

In contrast, FlowPersona models each user's historical usage behavior and service preference using weighted profile functions. The system then classifies flows using an APSO-optimized CNN, generating adaptive flow rules aligned with each user's actual service context. This enables priority-aware flow control-for example, assigning higher priority to User3's interactive video traffic-while maintaining fairness and stability across the network. As a result, FlowPersona improves overall QoS assurance and system responsiveness in dynamic SDN-IoT environments.

Flow persona is a QoS flow rule scheme that incorporates user attributes and business requirements. It involves collecting and analyzing network flow data F to create a persona for QoS flows. Based on this persona, QoS flow rules are formulated to ensure an optimal user network experience and enable fine-grained service quality management of QoS flows. Specifically, as depicted in Fig. 2, the model comprises four main components: user information collection, service category classification, QoS flow classification, and algorithm processing.

The persona gathers basic user information and stores it on the server. Its purpose is to classify QoS flow in real-time by analyzing user behavior preferences and historical data for various services.

This paper examines three service categories: video streaming, audio streaming, and ordinary data streaming. Each category has distinct network quality requirements. For instance, video streaming is highly sensitive to bandwidth and delay due to its demand for high-quality images, while audio streaming, despite lower bandwidth needs, is very delay-sensitive and jitter-sensitive. In contrast, ordinary data stream-

ing, like email and basic web browsing, has low real-time demands but requires data integrity. The QoS flow rule scheme proposed in this study addresses the specific needs of each media type during network transmission, ensuring service quality and enhancing the user experience. Specifically, we identified three distinct user types based on analysis of user behavior preferences and historical data. As indicated by Eq. 2, each user's feedback data is weighted differently. $User_1$ shows a preference for browsing the Internet, social media, and other ordinary data streaming services. $User_2$ engages more in activities like listening to music, making voice calls, and other audio streaming services. On the other hand, $User_3$ is inclined towards video streaming services such as video calls and watching videos. We quantify user preference for each service using $ub_{pre}^{service} = \frac{\text{The number of clicks of the service}}{\text{Total clicks}}$. Historical data $h_{data}^{service}$ reflects the number of hours users have spent on different services in the past 10 days. Additionally, we use $sr_{data} = 0.6 * \text{Service response time}(ms) + 0.4 * \text{Throughput}(Mbps)$ as the value of the service requirement data.

$$User_1 = 0.3 * (ub_{pre}^{video} + h_{data}^{video}) + 0.1 * (ub_{pre}^{audio} + h_{data}^{audio}) + 0.6 * (ub_{pre}^{ordinary} + h_{data}^{ordinary}) + sr_{data} \quad (2a)$$

$$User_2 = 0.3 * (ub_{pre}^{video} + h_{data}^{video}) + 0.6 * (ub_{pre}^{audio} + h_{data}^{audio}) + 0.1 * (ub_{pre}^{ordinary} + h_{data}^{ordinary}) + sr_{data} \quad (2b)$$

$$User_3 = 0.6 * (ub_{pre}^{video} + h_{data}^{video}) + 0.3 * (ub_{pre}^{audio} + h_{data}^{audio}) + 0.1 * (ub_{pre}^{ordinary} + h_{data}^{ordinary}) + sr_{data} \quad (2c)$$

The algorithm processing involves two aspects: real-time QoS flow problem, and QoS flow classification. For real-time processing, the NetFlow protocol is used to gather QoS-related data in real-time, and the ARIMA model enhances prediction accuracy through deep analysis. To ensure responsiveness in dynamic environments, the ARIMA model is periodically executed at fixed time intervals (every 5 seconds) over aggregated traffic statistics rather than individual flows. It forecasts key QoS metrics such as bandwidth demand and latency variation. These predictions are not used directly for path selection, but are instead used to update the QoS attribute weights in the Q' matrix, guiding pre-classification before CNN inference. To reduce runtime complexity, the ARIMA model is implemented using a fixed sliding window and precomputed coefficients. The Levinson-Durbin algorithm is employed to ensure numerical stability and efficient parameter estimation. This lightweight design ensures that the prediction process operates in real time at the SDN controller without adding delay to the flow rule generation process. This control enables integration of heterogeneous IoT services under a unified QoS-aware framework. In QoS flow classification, a QoS attribute matrix is constructed to reflect QoS demands, and the decision model is optimized to maximize QoS attribute values. The classification model utilizes the APSO algorithm to adjust the hyperparameters of a one-dimensional CNN. By training and optimizing these hyperparameters, the model finds the optimal configuration, reducing the labor cost of manual adjustments, improving classification accuracy, and enhancing the speed and performance of the neural network.

As shown in Fig 3, the computed user personas are incorporated into the system workflow through the weighting of QoS attributes, forming the Q' matrix. This matrix, after normalization, is fed into the APSO-optimized CNN model to perform flow classification. The classification result is then used by the SDN controller to assign flow priorities and generate appropriate QoS flow rules. Since persona vectors are updated continuously based on user behavior, the overall process supports adaptive flow rule adjustment in real time.

3.3. Problem Formulation of QoS Attributes

Definition 1: End-to-end delay

In this paper, we replace delay with end-to-end delay between SR nodes, where SR nodes play a pivotal role as relays in wireless Ad hoc networks. It measures the total delay encountered by packet transmission from the source node to the destination node. Specifically, we define end-to-end delay q_{de} as the time elapsed from when the packet enters the queue buffer of the source node to when it reaches the end of the queue buffer of the target node. This delay encompasses various components, including transmission delay, queuing delay, and propagating delay.

To model the end-to-end delay between SR nodes, we utilize the M/M/1 queue model [27]. This model assumes that the arrival time follows a Poisson distribution, the service time follows an exponential distribution, and there is only a single service desk. Based on this model, q_{de} can be calculated using the following formula:

$$q_{de}(t) = T_{queue}(t) + T_{transmission}(t) + T_{propagation}(t) \quad (3)$$

Among them, $T_{queue}(t)$ represents the queuing delay at time t , which is the time a packet waits in the queue before processing. $T_{transmission}(t)$ denotes the transmission delay at time t , the time needed to send the packet from the source to the destination. $T_{propagation}(t)$ indicates the propagation delay at time t , which is the time required for the packet to travel through the channel.

The queuing delay $T_{queue}(t)$ can be modeled using the M/M/1 queue model. In this model, the average queue time $Q_u(t)$ at time t and the packet arrival rate $\lambda(t)$ to the network are related to the packet processing and transmission rate $\mu(t)$. This relationship can be expressed as:

$$Q_u(t) = \frac{\lambda(t) * p(t)}{\mu(t) - \lambda(t)} \quad (4)$$

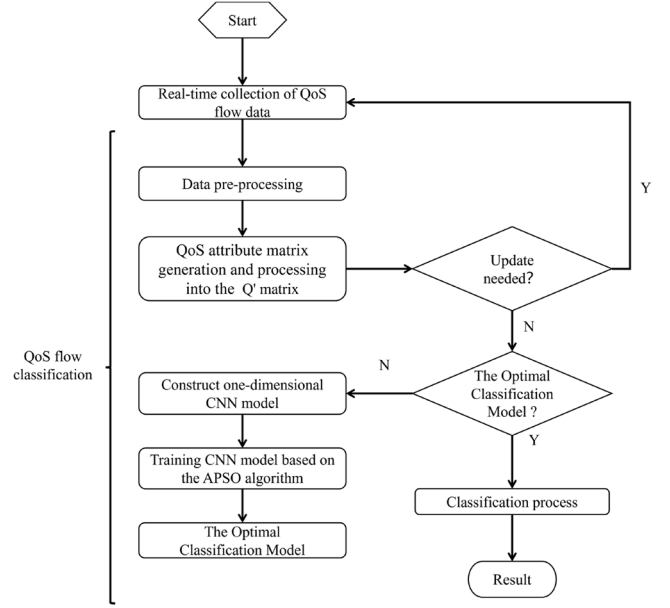


Fig. 3. Flowchart of algorithm.

Where $p(t) = \frac{\mu(t) - \lambda(t)}{\mu(t)}$ represents the probability that the SR node cannot process or transmit the data packet in time at moment t . The average queuing time $Q_u(t)$ at time t can be used to determine the queuing delay $T_q(t)$ at the same moment, with the formula as follows:

$$T_{queue}(t) = Q_u(t) / \lambda(t) \quad (5)$$

In summary, we can model q_{de} by combining queuing theory and the Poisson process. By utilizing the above formula, we define and calculate the end-to-end delay between SR nodes.

Definition 2: Throughput

Throughput refers to the quantity of data transmitted over a specific path or connection within a time unit. It is a crucial metric for assessing the efficiency of network transmission, as it evaluates the speed and capacity of data transmission within the network. The magnitude of throughput directly impacts user experience and the extent to which business needs are satisfied. A higher throughput signifies a network's ability to transfer data at a faster rate, providing enhanced data transfer rates and efficiency [28].

Hence, we require a specific formula to define throughput q_{th} :

$$q_{th}(t) = \sum_{i \in \mathcal{L}} T h_i(t) \quad (6)$$

Definition 3: Packet loss rate

The packet loss rate signifies the percentage or decimal representation of packets that fail to reach the receiver during transmission. It serves as a measure of the proportion of lost packets in data transmission.

$$q_{lo}(t) = \frac{N_i(t)}{N_s(t)} \quad (7)$$

Where N_i represents the number of lost packets, referring to the packets that did not successfully reach their destination during transmission. N_s represents the total number of packets sent within the same time period.

4. Algorithm

In this section, we will provide a comprehensive overview of the key algorithms discussed in this article. The overall algorithm flow chart is illustrated in Fig. 3.

It is important to note that the machine learning component of Flow Persona operates in an offline training mode, where NS-3 is employed to simulate the SDN-IoT topology and generate flow-level data for model training and validation. Once the CNN model optimized by APSO is trained, the learned classification and prediction functions are integrated with the SDN controller for real-time QoS flow rule enforcement. In this way, NS-3 provides a realistic emulation environment for data generation and performance validation, while the Flow Persona framework remains modular, allowing future integration of the ML model directly into NS-3 or other online platforms if required.

4.1. Real-time QoS Flow Problem

Real-time QoS flow is a critical issue in the flow persona model. Due to data processing delays and the dynamic nature of QoS attributes in SR nodes, the QoS flow rule scheme must rapidly converge to support real-time network flow monitoring and control. To address this, as shown in Algorithm 1, we employ the ARIMA statistical algorithm to handle real-time QoS flow, ensuring effective real-time monitoring and control of network flow.

Algorithm 1 Real-time QoS Flow Optimization.

- 1: **Input:** NetFlow data, QoS attribute time series
- 2: **Output:** Optimized real-time QoS flow rules
- 3: **Step 1: Data Collection and Preprocessing**
- 4: Activate NetFlow protocol, configure SolarWinds for real-time data collection
- 5: Perform linear interpolation for missing data and apply moving average for outlier smoothing
- 6: Normalize data to [0, 1] range
- 7: **Step 2: Stationarity Check**
- 8: Apply ADF test: $\xi_{y_t} = \alpha + \beta t + \gamma y_{t-1} + \sum_{i=1}^k \eta_i \xi_{y_{t-i}} + \varepsilon_t$
- 9: **if** significance level > 0.05 **then**
- 10: Apply differencing: $y'_t = y_t - y_{t-1}$
- 11: **end if**
- 12: **Step 3: ARIMA Parameter Identification**
- 13: Plot ACF and PACF to estimate p and q values
- 14: Use AIC criteria to select the best ARIMA model: $AIC = 2k - 2 \ln(L)$
- 15: **Step 4: Initial Model Fitting**
- 16: Fit ARIMA model to the time series:

$$y_t = c + \sum_{i=1}^p \phi_i y_{t-i} + \sum_{j=1}^q \theta_j \varepsilon_{t-j} + \varepsilon_t$$

- 17: **Step 5: Iterative Model Update**
- 18: **while** new data is collected or error threshold is exceeded **do**
- 19: Collect additional QoS flow data in real-time
- 20: Preprocess the newly collected data
- 21: Recheck stationarity and apply differencing if needed
- 22: Reidentify ARIMA parameters (p, d, q) using ACF, PACF, and AIC
- 23: Refit the ARIMA model with updated parameters and new data:

$$y_t = c + \sum_{i=1}^p \phi_i y_{t-i} + \sum_{j=1}^q \theta_j \varepsilon_{t-j} + \varepsilon_t$$

- 24: Forecast QoS time series values using the updated model
- 25: Evaluate forecast accuracy and update QoS flow rules dynamically
- 26: **end while**
- 27: **Step 6: Forecasting and Evaluation**
- 28: Forecast future QoS time series values using the latest model:

$$\hat{y}_{t+h} = c + \sum_{i=1}^p \hat{\phi}_i \hat{y}_{t+h-i} + \sum_{j=1}^q \hat{\theta}_j \hat{\varepsilon}_{t+h-j}$$

- 29: Evaluate final forecast accuracy using MSE and MAE
 - 30: Adjust network resources based on predictions
 - 31: **End**
-

The ARIMA(p, d, q) model combines Autoregressive model (AR) and Moving Average (MA) models to predict time series [29]. We can use historical time series data to predict the current value using the AR model,

apply the Integral (I) component for differencing non-stationary time series to transform them into stationary series, and correct the prediction result using the MA of historical prediction errors [30]. This approach, with its ability to capture short-term changes, is well-suited for modeling QoS attribute variations over time, enabling real-time analysis and prediction [31].

4.1.1. Data collection and preprocessing

Enable NetFlow on network routers and configure SolarWinds to collect and analyze data flow, calculating real-time time series of QoS flow. Use linear interpolation to fill in missing data points and apply moving average to smooth outliers. Finally, normalize and scale the data to the range [0, 1] to ensure consistency.

4.1.2. Determine the stationarity of time series

Use the Augmented Dickey-Fuller (ADF) test to compute ADF statistics with the formula (8): transmission. It serves as a measure of the proportion of lost packets in data transmission.

$$\xi_{y_t} = \alpha + \beta t + \gamma y_{t-1} + \sum_{i=1}^k \eta_i \xi_{y_{t-i}} + \varepsilon_t \quad (8)$$

where ξ_{y_t} is the differenced time series, α is the constant term, βt is the time trend term (if applicable), γy_{t-1} is the lag term, η_i are the coefficients of the differenced terms, and ε_t is the error term. The significance level from the ADF test determines the stationarity of the QoS flow time series. If the significance level is below 0.05, the time series is stationary; otherwise, differencing is required. For differencing, calculate $y'_t = y_t - y_{t-1}$, where y'_t is the differenced series value at time t , and y_t is the original series value at time t . Differencing removes the trend component, making the series more stationary.

4.1.3. Determine the parameters of the ARIMA model

Plot the Autocorrelation Function (ACF) and Partial Autocorrelation Function (PACF) graphs to preliminarily determine the ranges for the autoregressive order p and moving average order q based on the truncation and trailing of the autocorrelation coefficients. Construct ARIMA models for various p and q values, and calculate the Akaike Information Criterion (AIC) using (9):

$$AIC = 2k - 2 \ln(L) \quad (9)$$

where k is the number of model parameters and L is the model's likelihood function. Select the model with the lowest AIC value as the best ARIMA model.

4.1.4. Model fitting and iteration

Use the Levinson-Durbin algorithm to accurately calculate the autocorrelation and partial autocorrelation coefficients, with the formula (10). The algorithm is a recursive method for solving linear predictive coding problems using Yule-Walker equations, represented by a Toeplitz matrix with rows as shifted autocorrelation functions. It begins with a first-order AR model and progressively increases the order until the desired model order is achieved.

$$a_i = \frac{r_i - \sum_{j=1}^{i-1} a_j r_{i-j}}{e_{i-1}} \quad (10)$$

where a_i is the regression coefficient at step i , r_i is the autocorrelation coefficient, and e_{i-1} is the prediction error. We then use the optimal p and q parameters to fit the QoS flow time series data. The fitted ARIMA model is:

$$y_t = c + \sum_{i=1}^p \phi_i y_{t-i} + \sum_{j=1}^q \theta_j \varepsilon_{t-j} + \varepsilon_t \quad (11)$$

where c is a constant, ϕ_i are the autoregressive coefficients representing the influence of past values, θ_j are the moving average coefficients representing the influence of past prediction errors, and ε_{t-j} are the error term of the past j moment.

To adapt to changing QoS requirements, the model is re-fitted when new data arrives. Periodic error checks and performance evaluations trigger automatic updates, allowing the ARIMA model to adjust to network flow changes continuously [32],[33]. Each new data batch prompts a model update based on the latest information, with the QoS flow data collected via the NetFlow protocol and pre-processed accordingly. Updates occur either based on a set frequency or when an error threshold is exceeded. When triggered, the ARIMA model is refitted and its parameters are optimized to align with current QoS needs. The updated model then forecasts future QoS flow, enabling dynamic adjustments to QoS flow rules and resource allocation strategies to maintain real-time performance and prediction accuracy.

4.1.5. Prediction and real-time evaluation

Use the ARIMA model to predict future QoS flow values with the formula (12):

$$\hat{y}_{t+h} = c + \sum_{i=1}^p \hat{\phi}_i \hat{y}_{t+h-i} + \sum_{j=1}^q \hat{\theta}_j \hat{\varepsilon}_{t+h-j} \quad (12)$$

where $\hat{y}_{t+h}$ is the predicted value of step $t+h$, and $\hat{\varepsilon}_{t+h-j}$ represents the prediction error. To ensure consistently high prediction accuracy, the model's performance is re-evaluated after each data update. The accuracy of the predictions is evaluated by comparing them with actual data and calculating the mean squared error and mean absolute error. Based on the prediction results, the network resource allocation strategy is dynamically adjusted to optimize real-time QoS flow.

The ARIMA statistical algorithm effectively addresses real-time QoS flow issues by enabling timely network flow monitoring and control. Its advantage lies in learning time patterns and trends from historical data, allowing accurate prediction of future QoS values. This capability facilitates rapid convergence of the QoS flow rule scheme and precise real-time monitoring and control in the network environment.

4.2. QoS Flow Classification

QoS flow classification is an important part of the flow persona model, which is used to classify and identify network flow data so as to formulate appropriate rules and strategies for QoS flow. In this part, in order to realize QoS flow classification, we adopt a fine-grained QoS flow classification method [34] to form a QoS flow rule scheme.

4.2.1. Data Pre-processing

In the data pre-processing stage, we process and prepare the original network flow data to facilitate subsequent service classification based on QoS attribute values.

First, a significant volume of flow data is collected from the network, with a collection interval of t seconds. The data is then subjected to cleaning procedures, eliminating outliers, noise, and invalid data to ensure data quality and accuracy. Subsequently, the QoS attribute value corresponding to each flow data is calculated using the formula introduced in Section 3.3. This generates the QoS Attribute Matrix, which captures the values of the QoS attributes (as described in Section 4.2.2). Next, the data is standardized, unifying the value range of different features. This standardization is essential for achieving high-fine-grained classification and facilitates training and classification tasks for the CNN model. Lastly, the dataset is divided into training and validation sets. Notably, during CNN model training, it is necessary to label the type of flow data to which each sample belongs: video streaming, audio streaming, or ordinary data streaming. After the data cleaning process, the network flow relevant to the target task is filtered, aiming to provide comprehensive network state information that facilitates the training of the CNN model.

4.2.2. QoS Attribute Matrix

In [35], the author presents an effective approach to meeting users' QoS requirements by aggregating the QoS attributes of each service in

the service portfolio into a comprehensive QoS metric value. Furthermore, in [36], the author introduces the MQRP method, which optimizes decisions to maximize the values of QoS attributes within the network. Taking into account the unique requirements and limitations of satellite networks, this method determines routing paths and resource allocation strategies by considering trade-offs and optimizing various QoS attributes. Drawing inspiration from the concepts presented in these two papers, we have devised a QoS attribute matrix to be utilized in the flow persona model.

We define the matrix (13) as the QoS matrix.

$$Q = \begin{bmatrix} Q(1,1) & Q(1,2) & Q(1,3) \\ Q(2,1) & Q(2,2) & Q(2,3) \\ \vdots & \vdots & \vdots \\ Q(t,1) & Q(t,2) & Q(t,3) \\ \vdots & \vdots & \vdots \\ Q(1000,1) & Q(1000,2) & Q(1000,3) \end{bmatrix} \quad (13)$$

Where $Q(t,s)$ represents the value of the s -th QoS attribute at the t -th moment. $Q(t,1) = q_{de}(t)$, $Q(t,2) = q_{th}(t)$, $Q(t,3) = q_{lo}(t)$. As a result, the first column of the matrix represents the QoS attribute value of End-to-end delay at time t , the second column represents the QoS attribute value of throughput at time t , and the third column represents the QoS attribute value of packet loss rate at time t . By organizing the collected flow data into this matrix format, we can conveniently analyze and process the data, while capturing and analyzing the temporal characteristics of QoS attributes. Taking into account the complexity of subsequent CNN model training, we gathered 1000 instances of flow data to alleviate the challenges associated with data storage and computation. This approach helps to strike a balance between capturing sufficient data and avoiding an excessive collection of flow data, thus meeting the real-time requirements of QoS flow. Moreover, capturing 1000 instances of data allows us to sample at a moderate frequency during the actual classification process, providing a more accurate reflection of flow characteristics. At the same time, we define the user's weight matrix W as:

$$W = \begin{bmatrix} w_{de}^{user_1} & w_{th}^{user_1} & w_{lo}^{user_1} \\ w_{de}^{user_2} & w_{th}^{user_2} & w_{lo}^{user_2} \\ w_{de}^{user_3} & w_{th}^{user_3} & w_{lo}^{user_3} \end{bmatrix} \quad (14)$$

Where $w_{de}^{user_i}$ indicates the delay weight of user i , $w_{th}^{user_i}$ indicates the throughput weight of user i , and $w_{lo}^{user_i}$ indicates the packet loss rate weight of user i .

We obtain the Q' matrix by multiplying Q with W . To normalize the value range of different features, each Q' matrix is normalized before constructing the training sample.

$$Q'_n(t,s) = \frac{Q'(t,s) - Q'_{\min}(t,s)}{Q'_{\max}(t,s) - Q'_{\min}(t,s)} \quad (15)$$

Among them, $Q'_n(t,s)$ denotes the normalized values, $Q'_{\min}(t,s)$ represents the minimum of the s -th Q' matrix values, and $Q'_{\max}(t,s)$ denotes the maximum of the s -th Q' matrix values.

By organizing this data in the form of a Q' matrix, it can be arranged based on time and attribute, facilitating analysis and processing. This arrangement is particularly useful when dealing with real-time QoS flow problem, as it enables the capture and analysis of temporal characteristics of QoS attributes. It allows for a better observation of patterns and trends in QoS attribute changes over time, and facilitates the extraction of valuable time series features, which is convenient to solve the QoS real-time problem.

Scalability within the FlowPersona framework is addressed in two complementary dimensions. First, at the network level, the SDN-IoT mesh topology enables seamless horizontal expansion by incorporating additional Segment Routing nodes, ensuring stable performance even under increasing device density. Second, at the ML model level, the APSO-optimized CNN is trained using flow data from multiple network scales, making the classifier robust to traffic heterogeneity. For model

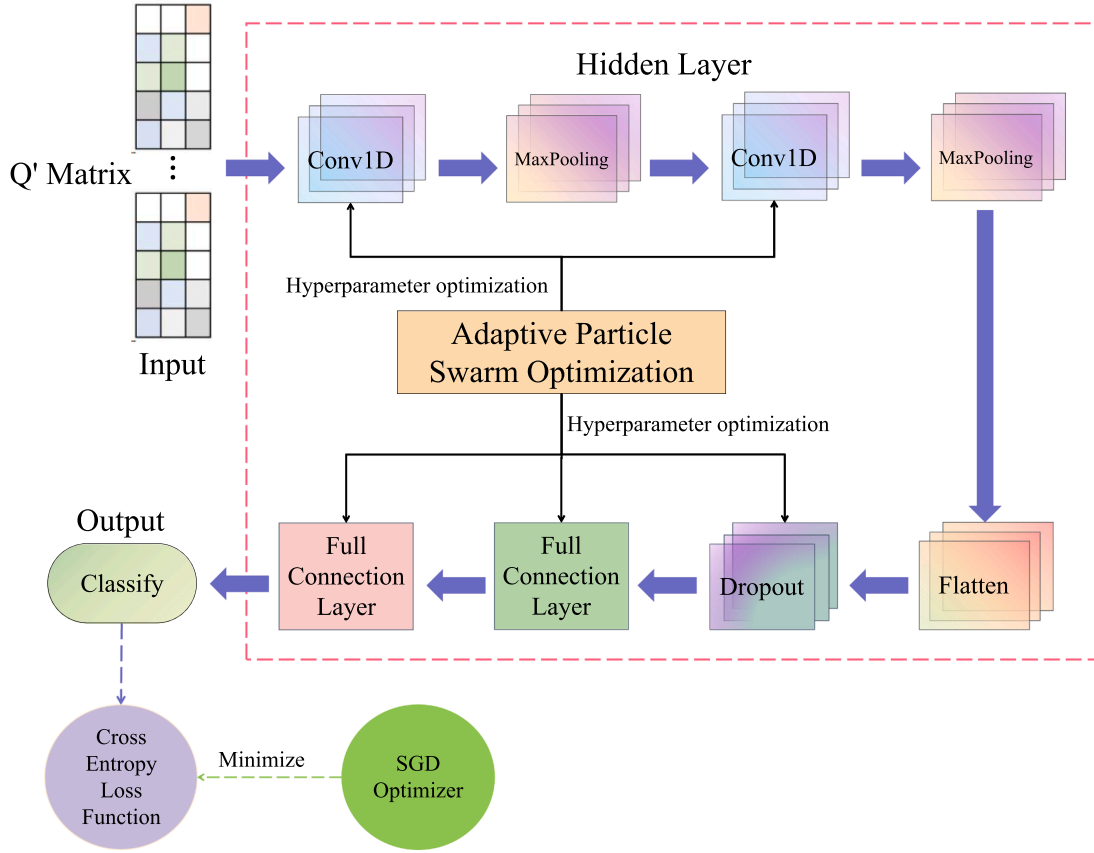


Fig. 4. Optimized CNN classification model based on APSO algorithm.

validation, the collected dataset is partitioned into training, validation, and testing sets, with k-fold cross-validation applied to mitigate overfitting. This design ensures that Flow Persona can generalize well to unseen data and maintain accuracy as the IoT platform scales. Furthermore, its modular structure allows incremental retraining when new flow patterns appear, thereby sustaining long-term scalability.

4.2.3. Framework

This paper adopts a classification model that optimizes the hyperparameter structure of a one-dimensional CNN using an adaptive particle swarm. By training the model and optimizing the hyperparameters of the neural network, suitable hyperparameters can be found. This approach avoids the high labor cost of manually adjusting hyperparameters to find a model suitable for the QoS flow classification task. Consequently, it improves the accuracy of classification, as well as the training speed and effectiveness of the neural network. The specific classification model of the CNN, optimized based on the APSO algorithm, is illustrated in Fig. 4.

One-dimensional CNN model construction: CNN is a multi-layer neural network where each layer consists of multiple two-dimensional planes [37]. Neurons in each layer compute their output by weighting and summing the elements from the previous layer and applying an activation function. In this study, we use a one-dimensional CNN to classify QoS attribute flows Algorithm 2.

As we consider Q' as the model's input and treat the flow data as a time series where each observation represents different types of flow data at each time point, the input data is transformed into a two-dimensional array. The dimensions of the input layer are (1000, 3), where each sample represents an instance of network flow. The one-dimensional convolution process is depicted in Fig. 5.

The hidden layer comprises 6 layers, each serving unique functions and producing distinct effects. The convolution layer extracts deeper and higher-level features by performing convolutions on pre-processed

Algorithm 2 The algorithm for finding the optimal hyperparameters of CNN model.

- 1: **Input:** Q' matrix
- 2: **Output:** The optimal hyperparameters
- 3: **Initialize:** Vectorize the parameters and use binary encoding as the position of particles; Randomly select a set of particles as the initial positions and velocities of the particle swarm, with the initial velocities generated using random numbers;
- 4: **while** $i \leq G$ **do**
- 5: Calculate the fitness value F for each particle, choose F_{min} , F_{avg} , and F_{cur} ;
- 6: Categorize the state by f 's value, and adaptive control strategies are chosen accordingly.
- 7: Update the inertia weight ω ;
- 8: Update the $pbest_{ik}^i$ and $gbest_k^i$ according to the fitness value F ;
- 9: Update the velocity v_{ik}^{i+1} and position x_{ik}^{i+1} of the particles according to (24) and (25);
- 10: $i = i + 1$;
- 11: **end while**

data. Furthermore, the feature planes are flattened before connecting to the subsequent layer through the second pooling layer. To mitigate potential overfitting during training, dropout is implemented to improve the network's generalization ability in each training session [38]. As a result, certain neurons cease to function with a specific probability, leading to different active neuronal nodes in each training batch and cycle. The dimensions of the output layer are also (1000, 3), with 3 representing the classification result, i.e., the three service categories (also the number of output nodes). Moreover, we assign different weights according to the importance of each QoS attribute value

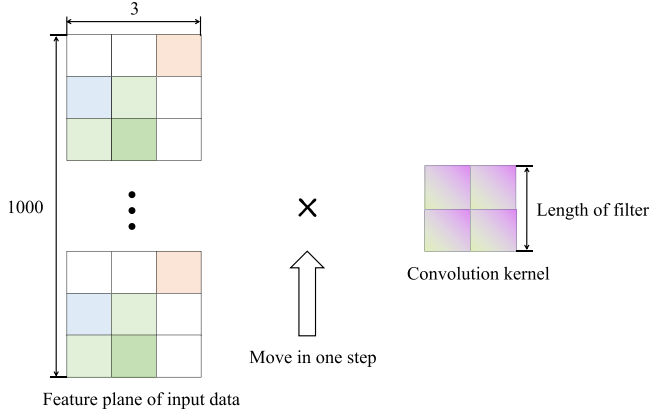


Fig. 5. One-dimensional convolution process.

to each of the three service categories to ensure that the classification results align as closely as possible with the actual outcomes. For hyperparameters such as weight, bias, and learning rate that require optimization in the model, we define them as unknown variables to be determined.

Training the hyperparameters of the CNN model base on the APSO algorithm: Adaptive Particle Swarm Optimization is a heuristic optimization algorithm proposed by Zhan *et al.* [39]. As a derivative optimization model of the Particle Swarm Optimization (PSO) algorithm, the APSO algorithm offers enhanced optimization performance. The evolution factor (f) is used to assess the state of the particle swarm and dynamically adjust the weight based on this state, improving global search capability. In the training process of the standard PSO algorithm, if the inertia weight ω is set to a large value, particles can easily escape local optima, but it may hinder iterative convergence in later stages of training. Conversely, if the inertia weight ω is set to a small value, particles tend to get trapped in locally optimal solutions [40]. The APSO algorithm addresses the negative impact of random initialization speed on particle optimization by dynamically adjusting the inertia weight, simplifying the algorithm and accelerating convergence. By using APSO to optimize the hyperparameters of a CNN and integrating QoS flow classification requirements during training, both training time and labor costs are significantly reduced compared to traditional manual parameter adjustment or static optimization methods. Furthermore, combining the ARIMA model with the APSO algorithm allows for the design of a real-time optimization strategy based on the dynamic characteristics of QoS flow, thereby enhancing the real-time classification capability of network flow.

We need to collect network flow data and apply principal component analysis (PCA) to reduce its dimensionality, then represent it as a tensor. We then split the dataset into two equal parts: one with both network flow data and classification labels, and another with only network flow data. The first part is used to train an untrained CNN model to obtain an optimal model for network flow classification.

The specific process of training the hyperparameters of the CNN model base on the APSO algorithm, as described in this paper, is outlined below:

Step 1: Coding and initialization of model hyperparameters. The adaptive particle swarm's global search capability is utilized to vectorize the hyperparameters of the CNN model that needs to be solved, with binary coding used for the particle position [41]. A group of particles is randomly selected to define the initial positions and velocities of the particle swarm, with their positions serving as the historical optimal positions. The initial velocities are set using random numbers. The acceleration coefficients c_1 and c_2 for the cognitive and social parameters are initialized to $c_1 = c_2 = 2.0$. The maximum and minimum inertia weights are set to $\omega_{\max} = 0.9$ and $\omega_{\min} = 0.9$, respectively.

Step 2: Iterative updating of the particle swarm. Based on the business scenario and network environment, we employ the cross-entropy loss function to evaluate model performance. Predictions are made using a feedforward neural network, and the loss function for the current weights is computed. The weights and biases are then updated using the error backpropagation algorithm based on the gradient of the loss function to minimize the loss value. During the first training cycle, we set the cross-entropy value of the CNN as the fitness function value of the particle [42]. We evaluate the fitness of all particles, selecting the one with the smallest fitness value as the global optimal particle. This method speeds up model training and minimizes the gap between actual and predicted classifications, as a lower cross-entropy loss reflects better prediction accuracy. Next, we compute the distance d_i between the global optimal particle and its nearest neighbor. We then compare the maximum $d_{\max}$ and minimum distances $d_{\min}$ among all particles, and calculate the average distance of particle i to all other particles using the Euclidean metric as shown in Eq. 16. This allows us to determine the evolution factor f .

$$d_i = \frac{1}{N-1} \sum_{j=1, j \neq i}^N \sqrt{\sum_{d=1}^D (x_{id}^k - x_{jd}^k)^2} \quad (16)$$

$$f = \frac{d_i - d_{\min}}{d_{\max} - d_{\min}} \quad (17)$$

Where N denotes the particle population size and the evolution factor $f \in [0, 1]$. Based on the value of f , fuzzy classification divides into four states: exploration state S_1 , development state S_2 , convergence state S_3 , and jump state S_4 . The formula for determining the classification value is as follows:

1. Exploration state S_1 :

$$\mu_{S_1}(f) = \begin{cases} 0 & \text{if } 0 \leq f \leq 0.4 \\ 5f - 2 & \text{if } 0.4 < f \leq 0.6 \\ 1 & \text{if } 0.6 < f \leq 0.7 \\ -10f + 8 & \text{if } 0.7 < f \leq 0.8 \\ 0 & \text{if } 0.8 < f \leq 1 \end{cases} \quad (18)$$

2. Development state S_2 :

$$\mu_{S_2}(f) = \begin{cases} 0 & \text{if } 0 \leq f \leq 0.2 \\ 10f - 2 & \text{if } 0.2 < f \leq 0.3 \\ 1 & \text{if } 0.3 < f \leq 0.4 \\ -5f + 3 & \text{if } 0.4 < f \leq 0.6 \\ 0 & \text{if } 0.6 < f \leq 1 \end{cases} \quad (19)$$

3. Convergence state S_3 :

$$\mu_{S_3}(f) = \begin{cases} 1 & \text{for } 0 \leq f \leq 0.1 \\ -5f + 1.5 & \text{for } 0.1 < f \leq 0.3 \\ 0 & \text{for } 0.3 < f \leq 1 \end{cases} \quad (20)$$

4. Jump state S_4 :

$$\mu_{S_4}(f) = \begin{cases} 0 & \text{for } 0 \leq f \leq 0.7 \\ 5f - 3.5 & \text{for } 0.7 < f \leq 0.9 \\ 1 & \text{for } 0.9 < f \leq 1 \end{cases} \quad (21)$$

Step 3: Hyperparameter optimization and weight updating. Firstly, the boundary for c_1 and c_2 is defined as follows: $|c_i(g+1) - c_i(g)| \leq \delta$, where $c_i(g)$ represents the value of c_i at the g -th iteration and $\delta \in [0.05, 0.1]$ represents acceleration. Since $c_1 + c_2 \leq 4.0$, if the sum of c_1 and c_2 exceeds 4.0, normalization is applied:

$$c_i = \frac{c_i}{c_1 + c_2} \times 4.0, \quad i = 1, 2 \quad (22)$$

Then, based on the classification results from Step 2, the acceleration coefficients c_1 and c_2 are adjusted adaptively: in the exploration state, c_1

increases by 0.05 and c_2 decreases by 0.05; in the development state, c_1 increases by 0.005 and c_2 decreases by 0.005; in the convergence state, both c_1 and c_2 increase by 0.005; and in the jump state, c_1 decreases by 0.05 while c_2 increases by 0.05. Finally, the inertia weight is updated using the formula as follows.

$$\omega = \begin{cases} \omega_{\min} - (\omega_{\max} - \omega_{\min}) \frac{F_{\text{cur}} - F_{\min}}{F_{\text{avg}} - F_{\min}}, & \text{if } F_{\text{cur}} \leq F_{\text{avg}} \\ \omega_{\max}, & \text{if } F_{\text{cur}} > F_{\text{avg}} \end{cases} \quad (23)$$

Here, F_{cur} represents the fitness function value of the current particle, F_{avg} denotes the average fitness function value of the current population, and $F_{\min}$ signifies the fitness function value of the globally optimal particle. This process ensures that each parameter gradually adjusts within the search space, moving toward a direction that allows for accurate classification based on QoS attribute values.

Step 4: Update particle position and velocity. The position and velocity of each particle are updated following the rules of the APSO algorithm. The velocity update formula is:

$$v_{im}^{k+1} = \omega v_{im}^k + c_1 \text{rand}_1(pbest_{im}^k) + c_2 \text{rand}_2(gbest_m^k - x_{im}^k) \quad (24)$$

The position update formula is:

$$x_{im}^{k+1} = x_{im}^k + v_{im}^{k+1} \quad (25)$$

Where $v_i^k = [v_{i1}^k, v_{i2}^k, v_{i3}^k, \dots, v_{iD}^k]$ denotes the velocity parameters, and $x_i^k = [x_{i1}^k, x_{i2}^k, x_{i3}^k, \dots, x_{iD}^k]$ denotes the position parameters. v_{im}^{k+1} represents the m -th component of the velocity of particle i at iteration $k+1$, and x_{im}^{k+1} represents the m -th component of the position of particle i at iteration $k+1$. D is the dimension of the particle swarm, rand_1 and rand_2 are two random numbers uniformly distributed in the interval $[0, 1]$. $pbest_{im}^k$ denotes the best position found by the current particle, and $gbest_m^k$ denotes the best position found by the entire particle swarm.

Step 5: Convergence Judgement and Termination. Repeat steps 2-4 until the convergence condition is met. Convergence is determined when the maximum number of iterations is reached, and the training process is terminated.

Step 6: Selecting the Optimal Solution. During the particle swarm's iterative process, the global optimal fitness function value and its corresponding model hyperparameters are tracked. The hyperparameters associated with the particle having the smallest fitness function value are selected as the global optimal solution and are used as the optimal hyperparameters for the APSO-CNN model. This model represents the optimal CNN model. The latter part of the dataset is then fed into this optimal CNN model, and the output matrix is processed through the softmax function to obtain the probability distribution of the network flow data's category. The category with the highest probability is determined to be the classification of the network flow data, thereby achieving network flow classification.

While the simulation setup in this work utilizes a mid-scale IoT-oriented topology, the FlowPersona architecture is designed to be topology-independent and scalable. The user persona construction and QoS classification rely on global traffic features and user behavior patterns, which do not grow proportionally with the number of nodes. Therefore, the computational cost remains stable under larger topologies such as GEANT or Internet2. In terms of flow diversity, the CNN classifier is trained using mixed service types (video, audio, ordinary) to ensure robust prioritization across concurrent heterogeneous flows. Moreover, although this paper does not focus on link or node failure recovery, FlowPersona's modular design allows seamless integration with fault-tolerant SDN control frameworks or routing resilience modules. These aspects are considered future work directions to enhance deployment robustness in real-world environments.

5. Experimental Results and Performance Evaluation

5.1. Setup

To simulate and evaluate the performance of the proposed flow persona model, we utilize an NS-3-based network simulator. The network module is implemented on the NS-3 platform to capture flowing data in three different service scenarios, creating datasets and calculating the corresponding QoS attribute values. Each service scenario corresponds to a specific traffic type-such as video, audio, or ordinary data-and is assigned distinct QoS attributes including delay sensitivity and bandwidth demand. Since we employ software simulation, real-time collection of packet loss rate is not feasible. Therefore, we randomly set the packet loss rate within the range of $[0.0001-0.1]$ [43]. Furthermore, the CNN model trained using the APSO algorithm is built on the TensorFlow deep learning framework.

For the labeled flow dataset consisting of 1000 instances, we divide it into two parts: 80% serves as the training set, and the remaining 20% is allocated as the validation set. Performance tests are conducted on an Intel Core i7 machine with 32 GB of RAM and an NVIDIA GeForce RTX 4060 GPU card. Additionally, in order to accommodate the flow persona model we constructed, we deploy 500 SR nodes in the network.

5.2. Discussion of Algorithm

Based on experimental data analysis, we assign different weights to each QoS attribute according to the importance for each service category [44],[45],[46]. For instance, U_{ser1} , which primarily uses video streaming, has the following weights: end-to-end delay weight $w_{de}^{U_{ser1}} = 0.29$, throughput weight $w_{th}^{U_{ser1}} = 0.57$, and packet loss rate weight $w_{lo}^{U_{ser1}} = 0.14$. U_{ser2} , which tends to use audio streaming media, has weights: $w_{de}^{U_{ser2}} = 0.45$, $w_{th}^{U_{ser2}} = 0.25$, and $w_{lo}^{U_{ser2}} = 0.30$. Lastly, U_{ser3} , which uses ordinary data streaming media, has weights: $w_{de}^{U_{ser3}} = 0.32$, $w_{th}^{U_{ser3}} = 0.43$, and $w_{lo}^{U_{ser3}} = 0.25$.

Regarding the CNN model trained using the APSO algorithm, the parameters are configured as follows: The maximum number of iterations for the particle swarm is set to 200, the population size is 30, the acceleration coefficient $c_1 = c_2 = 2$, the minimum inertia weight is $w_{\min} = 0.4$, and the maximum inertia weight is $w_{\max} = 0.9$. Considering precision and real-time requirements, the data collection interval during the training process is set to 0.25 seconds ($t = 0s, 0.25s, 0.50s, 0.75s, \dots$). Setting the data acquisition time to 0.25 seconds significantly improves real-time performance while reducing network bandwidth usage. Given the dynamic nature of the network, this collection interval exhibits strong adaptability to network jitter, thereby enhancing the network's robustness [47]. In the actual classification process using the optimized model parameters, all flow data is collected every minute, resulting in a data collection interval of $t = 60/1000 = 0.06s$. This balance between data precision and real-time requirements helps reduce data processing complexity.

Based on the results from [48] and [49], a comparative experiment of optimizers was conducted using the VGG + BN + Dropout network and the DenseNet architecture on the dataset. The experimental results indicate that SGD (stochastic gradient descent) performs the best in terms of test accuracy and convergence speed. Given that this paper involves a multi-classification task, we choose SGD as the optimization method for the model training process. To address the frequent oscillation issue of SGD at local optima, we introduce a first-order momentum setting of 0.9 and dynamic learning rate decay to improve the training effectiveness [42].

As depicted in Fig. 6, after 200 iterations of optimization, the particle achieved a minimum fitness value of $f_{\min} = f_{\text{best}} = 0.1398$. This value also represents the minimum loss function value achieved on the validation set after the initial training. By identifying the parameters corresponding to the minimum fitness function value, we can determine

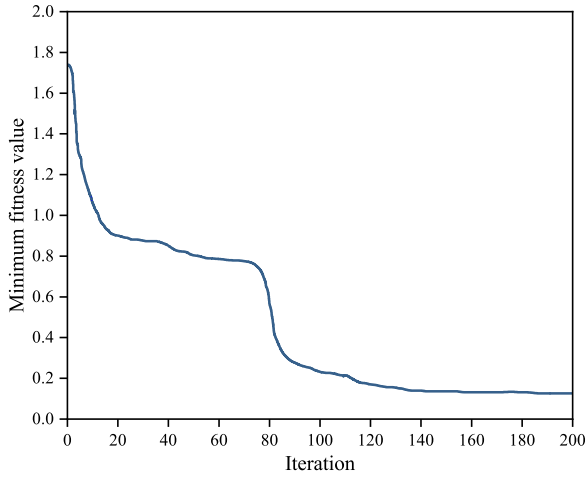


Fig. 6. Iterative results of particle.

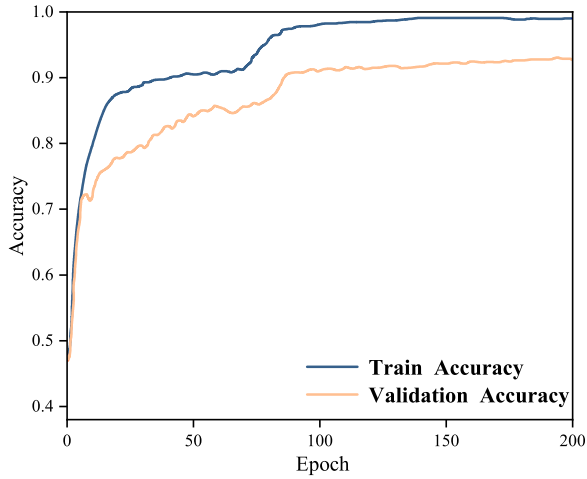


Fig. 7. Accuracy of flow persona in training set and validation set.

the optimal hyperparameters for the CNN model. Specifically, 62 convolution kernels in the first layer, 48 convolution kernels in the second layer, relu activation functions for both convolution layers, filters with a length of 2 in the first and second convolutional layers, a dropout rate of 0.25, 53 neurons in the first fully connected layer with relu activation, 35 neurons in the second fully connected layer with tanh activation, a batch size of 154, and a learning rate of 0.08.

These optimal hyperparameters are then incorporated into the CNN model, as illustrated in Fig. 4. The values of the cross-entropy loss function and accuracy for the training set, as well as the cross-entropy loss function and accuracy for the validation set, are obtained for each training cycle. Fig. 7 presents the accuracy of the training set and the validation set of the flow persona model using the optimal hyperparameters.

5.3. Performance Metrics

We assess the performance of the model on validation sets using previously unseen QoS flow data. To evaluate the accuracy of QoS flow classification results, we compare the predictions of the flow persona model with the actual labels. The following indexes are calculated for assessment: accuracy, precision, recall rate, and F-score.

Accuracy: It is defined as the ratio of the number of correctly classified QoS flows to the total number of QoS flows currently input

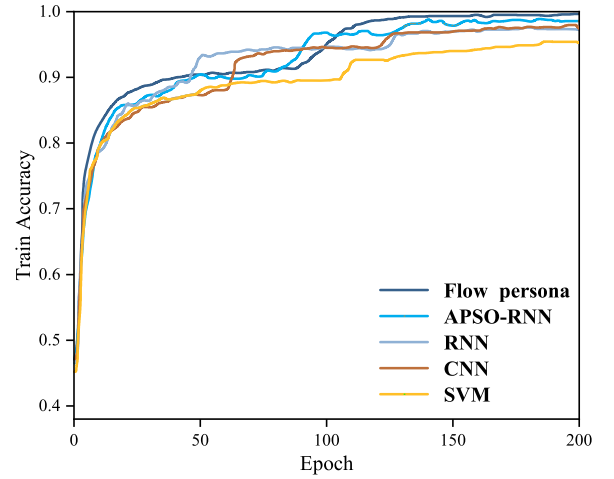


Fig. 8. Accuracy in training set.

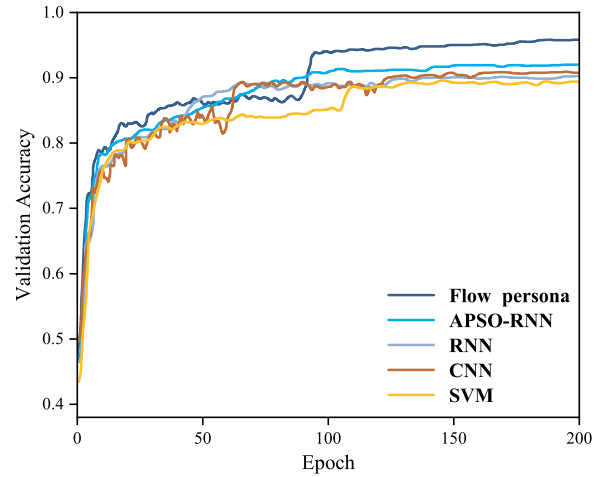


Fig. 9. Accuracy in validation set.

Precision(P): Calculated as the ratio of true positives T_p to the sum of true positives and false positives F_p for each service class:

$$P = T_p / (T_p + F_p) \quad (26)$$

Recall rate (R): For each category, it is defined as the ratio of true positives T_p to the sum of true positives and false negatives F_n :

$$R = T_p / (T_p + F_n) \quad (27)$$

F-score: It is defined as the harmonic mean of accuracy P and recall R , taking into account the performance of both:

$$F = ((1/P + 1/R)/2)^{-1} = 2RP / (R + P) \quad (28)$$

5.4. Performance Comparison

In this paper, we compare the flow persona model with the APSO-based RNN model, as well as the RNN, CNN, and SVM models, where the latter three directly classify the QoS flow without undergoing training through the APSO algorithm. The comparison results on the training set and the validation set are presented in Fig. 8 and Fig. 9.

It is evident that starting from epoch 91, the accuracy of the flow persona model showed a significant increase, surpassing the other four models. Moreover, a similar trend in accuracy was observed in the validation set. After 200 epochs of training, the flow persona model achieved an accuracy of approximately 94.6% on the validation set, while the other four models achieved around 90% accuracy. This indicates that the flow persona model has higher classification accuracy

Table 3
Performance comparison of five models.

Model	Precision	Recall	F-score
APSO-RNN	0.911	0.911	0.911
RNN	0.899	0.901	0.900
CNN	0.908	0.909	0.908
SVM	0.888	0.889	0.889
FlowPersona	0.936	0.936	0.936

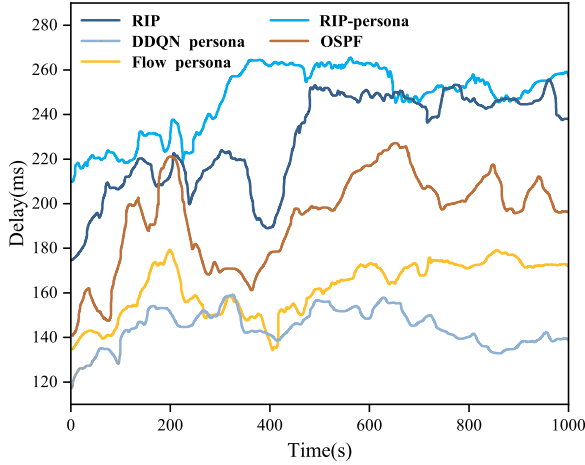


Fig. 10. Delay of the network under different algorithms.

and better overall classification performance. It demonstrates that the algorithm processing of the flow persona model is well-designed and has a positive impact on QoS flow classification, resulting in a well-performing QoS flow rule scheme. It also demonstrates that the APSO algorithm effectively promotes hyperparameter optimization of the CNN model and attains the global optimal solution.

Next, we evaluated these models in terms of precision P , recall rate R , and F-score to measure classification accuracy. As shown in Table 3, the flow persona model achieved a precision, recall rate, and F-score of 0.936. This indicates that the flow persona model can accurately classify positive and negative cases, better capturing the true positive cases and achieving a good balance between precision and recall rate. In comparison to the other four models, particularly the slightly superior APSO-RNN model. The flow persona model demonstrates a 2.7% improvement in precision, recall rate, and F-score. These results clearly indicate that the flow persona model achieves higher accuracy and overall performance in QoS flow classification.

We are not merely satisfied with the results of QoS flow classification. Additionally, we applied the traditional OSPF routing algorithm to route the data post-QoS classification, as shown in Fig. 10, Fig. 11 and Fig. 12. From Fig. 10, it is evident that the average delay after routing with the flow persona model is significantly better than that of the OSPF, RIP, and RIP-persona routing algorithms. This improvement is attributed to the QoS flow classification prior to routing. However, the DDQN-persona algorithm achieves an average delay of 139.2ms, outperforming the Flow Persona's 158.2ms. As shown in Fig. 11, the throughput value of the flow persona is moderate, and it can well provide different specific flows according to different service categories. This avoids the extremes of high or low throughput, which could otherwise lead to performance bottlenecks, degraded user experience, or network performance issues. In terms of average route computation time, the flow persona model is also slightly less efficient than DDQN-persona. Nonetheless, we believe that with the design of an effective AI routing algorithm specifically for flow persona, it can leverage real-time user and service requirements from persona for dynamic routing, potentially surpassing the performance of modern AI-based routing algorithms like DDQN-persona.

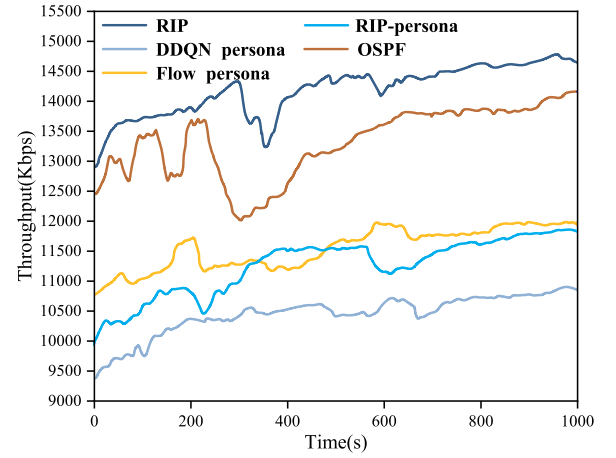


Fig. 11. Throughput of the network under different algorithms.

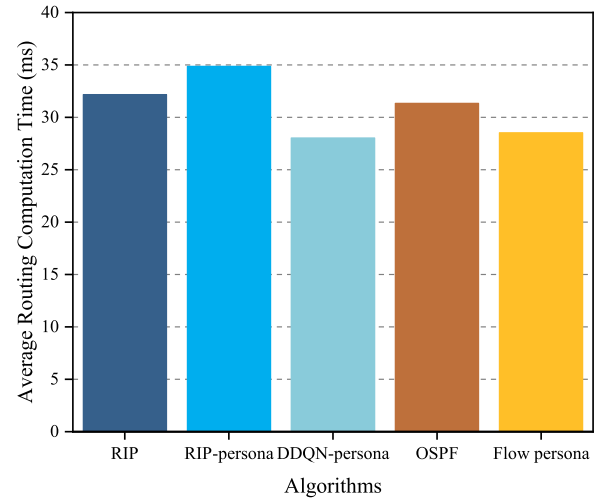


Fig. 12. Average routing computation time of all algorithms.

However, it should be noted that the convergence speed of the flow persona model for QoS flow classification is comparatively slower than that of the RNN and CNN models. According to the 'No Free Lunch theorem' [50], no algorithm can excel in all aspects or classes of problems. Therefore, enhancing the convergence speed of the flow persona model for QoS flow classification and developing an intelligent routing algorithm are key challenges that we aim to address in our future research.

6. Conclusion

In this paper, we propose a flow persona model to address the challenges of managing diverse data flow in SDN-IoT scenarios and meeting specific requirements. This model accurately classifies QoS flows and effectively targets them to different services.

In Section 3, a large number of SR nodes are deployed across the network. The flow persona model is divided into four main components: user information collection, service category classification, QoS flow classification, and algorithm processing. In the algorithm processing part, we address the real-time QoS flow issue using the ARIMA statistical algorithm. When training the CNN model with the APSO algorithm, the cross-entropy loss function value during the initial training period of the CNN is used as the fitness value for APSO. During the particle swarm iteration and optimization process, by updating the swarm's velocity and position, we identify the smaller fitness values, ultimately leading to the discovery of the optimal CNN model. Simulation results demonstrate that the flow persona model performs well in QoS flow

classification. Compared to traditional routing algorithms, the flow persona model offers significant advantages, highlighting its effectiveness.

In summary, the flow persona model possesses remarkable applicability and flexibility. This model can deliver effective QoS policies for various services in large-scale IoT deployments, leveraging diverse network resources for precise QoS flow scheduling. It is also capable of handling the future growth in network devices, ensuring optimal user experience and satisfaction. Additionally, the model has significant potential for real-world applications. For example, in smart cities, it can be integrated into IoT networks to optimize the operation of traffic lights, energy management systems, and emergency response mechanisms through accurate classification and efficient scheduling of critical traffic. This would enhance urban resource utilization and improve residents' quality of life. However, some limitations should be noted. In practical deployment, factors like device mobility and priority conflicts in large-scale networks need to be carefully addressed.

In the future, we will focus on efficiently managing flow rules, considering factors such as mobility, rule priority, overhead, and network capacity constraints. It will also involve further collection and analysis of flow statistics, exploring the applicability of algorithms across different network structures and scales, and investigating multi-agent cooperative training and planning methods based on reinforcement learning. Additionally, integrating heuristic algorithms will enhance the adaptability of the scheme and expand its potential for larger-scale and more complex application scenarios.

CRedit authorship contribution statement

Hao She: Writing – review & editing; **Lixing Yan:** Writing – original draft; **Xin An:** Writing – review & editing; **Chuanfeng Mao:** Writing – review & editing; **Yongan Guo:** Funding acquisition.

Data availability

The authors are unable or have chosen not to specify which data has been used.

Declaration of competing interest

No potential conflict of interest was reported by the authors.

References

- [1] X. Huang, M. Zeng, K. Xie, Intelligent Traffic Control For Qos Optimization In Hybrid Sdns 189 (2021) 107877.
- [2] W. Sun, Z. Wang, G. Zhang, A QoS-guaranteed intelligent routing mechanism in software-defined networks, *Computer Networks* 185 (2021) 107709. <https://doi.org/10.1016/j.comnet.2020.107709>
- [3] Z. Guo, Y. Shen, S. Wan, W.-L. Shang, K. Yu, Hybrid Intelligence-Driven Medical Image Recognition for Remote Patient Diagnosis in Internet of Medical Things, *IEEE Journal of Biomedical and Health Informatics* 26 (12) (2022) 5817–5828. <https://doi.org/10.1109/JBHI.2021.3139541>
- [4] A. Cooper, *The inmates are running the asylum*, Springer, 1999.
- [5] J. An, H. Kwak, S. Jung, J. Salminen, M. Admad, B. Jansen, Imaginary People Representing Real Numbers: Generating Personas from Online Social Media Data, *ACM Trans. Web* 12 (4) (2018). <https://doi.org/10.1145/3265986>
- [6] M.U. Farrukh, R. Wainwright, K. Crockett, D. McLean, N. Dagnall, Building Actionable Personas Using Machine Learning Techniques, in: 2022 IEEE Symposium Series on Computational Intelligence (SSCI), IEEE, 2022, pp. 463–472.
- [7] Y. Luo, P. Liu, E.K. Choe, Co-Designing Food Trackers with Dietitians - Identifying Design Opportunities for Food Tracker Customization (2019) 592–13.
- [8] L. Wang, L. Li, H. Cai, L. Xu, B. Xu, L. Jiang, Analysis of regional group health persona based on image recognition, in: 2018 Sixth International Conference on Enterprise Systems (ES), IEEE, 2018, pp. 166–171.
- [9] J. Salminen, S.-G. Jung, S. Chowdhury, D.R. Robillos, B. Jansen, The ability of personas: An empirical evaluation of altering incorrect preconceptions about users 153 (2021) 102645.
- [10] P. Ngantcha, M.T. Amith, K. Roberts, J.A. Valenza, M. Walji, C. Tao, Dental EHR-infused Persona Ontologies to Enrich Dental Dialogue Interaction of Agents 2021 (2021) 1818–1825.
- [11] Y. Cao, W. Bi, M. Fang, S. Shi, D. Tao, A Model-Agnostic Data Manipulation Method for Persona-based Dialogue Generation Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers) (2022) 7984–8002.
- [12] W. Shuai, D. Yuning, L. Tao, Network Traffic Classification Based on LSTM and Feature Generation, *Journal of Applied Sciences* 40 (05) (2022) 758–769.
- [13] J. Prados-Garzon, T. Taleb, M. Bagaa, Optimization of Flow Allocation in Asynchronous Deterministic 5G Transport Networks by Leveraging Data Analytics 22 (2021) 1–1.
- [14] G. Hu, X. Xiao, M. Shen, B. Zhang, X. Yan, Y. Liu, TCGNN: Packet-grained network traffic classification via Graph Neural Networks 123 (2023) 106531.
- [15] J. Zhang, F. Li, F. Ye, Sustaining the High Performance of AI-Based Network Traffic Classification Models 31 (2023) 816–827.
- [16] M. Ibrar, L. Wang, N. Shah, O. Rottenstreich, G.-M. Muntean, A. Akbar, Reliability-Aware Flow Distribution Algorithm in SDN-Enabled Fog Computing for Smart Cities 72 (2023) 573–588.
- [17] L. Xingtong, Z. Hong, H. Jianhua, An Improved Proximal Optimization Multi-objective Flow QoS Scheduling Strategy, *Journal of Applied Sciences* 42 (03) (2024) 499–512.
- [18] N. Saha, S. Misra, S. Bera, Q-flag: QoS-aware flow-rule aggregation in software-defined IoT networks, *IEEE Internet of Things Journal* 9 (7) (2021) 4899–4906.
- [19] S. Paramasivam, R. Leela Velusamy, J.V. Nishaanth, Categorical learning for automated network traffic categorization for future generation networks in SDN, *Computing* 106 (5) (2024) 1451–1473. <https://doi.org/10.1007/s00607-024-01277-y>
- [20] S. Xie, G. Hu, C. Xing, Y. Liu, Online Elephant Flow Prediction for Load Balancing in Programmable Switch-Based DCN, *IEEE Transactions on Network and Service Management* 21 (1) (2024) 745–758. <https://doi.org/10.1109/TNSM.2023.3318752>
- [21] J. Zhang, F. Li, F. Ye, Sustaining the High Performance of AI-Based Network Traffic Classification Models, *IEEE/ACM Transactions on Networking* 31 (2) (2023) 816–827. <https://doi.org/10.1109/TNET.2022.3203227>
- [22] J. Zhao, Q. Li, Y. Hong, M. Shen, MetaRockETC: Adaptive Encrypted Traffic Classification in Complex Network Environments via Time Series Analysis and Meta-Learning, *IEEE Transactions on Network and Service Management* (2024).
- [23] X. Wang, W. Wei, X. Yu, D. Zheng, N. Kuma, L. Liu, Ensemble Learning-based Traffic Classification with Small-Scale Datasets for Wireless Networks, in: IEEE INFOCOM 2024-IEEE Conference on Computer Communications Workshops (INFOCOM WK-SHPS), IEEE, 2024, pp. 1–6.
- [24] X. Wang, W. Wei, C. Liu, X. Zhang, W. Lu, Y. Zhong, Network Traffic Classification with Small-Scale Datasets Using Ensemble Learning, in: ICC 2024-IEEE International Conference on Communications, IEEE, 2024, pp. 1–6.
- [25] Y.S.K. Manjunath, S. Zhao, X.-P. Zhang, L. Zhao, Time-Distributed Feature Learning for Internet of Things Network Traffic Classification, *IEEE Transactions on Network and Service Management* (2024).
- [26] M. Farhan, N. Shah, L. Wang, G.-M. Muntean, H.H. Song, RDG-TE: Link reliability-aware DRL-GNN-based traffic engineering in SDN, *Expert Systems with Applications* 265 (2025) 125963.
- [27] T.-N. Tran, T.-V. Nguyen, K. Shim, D.B. da Costa, B. An, A Deep Reinforcement Learning-Based QoS Routing Protocol Exploiting Cross-Layer Design in Cognitive Radio Mobile Ad Hoc Networks, *IEEE Transactions on Vehicular Technology* 71 (12) (2022) 13165–13181.
- [28] D.A. Menasce, QoS issues in web services, *IEEE internet computing* 6 (6) (2002) 72–75.
- [29] Y. Xie, M. Jin, Z. Zou, G. Xu, D. Feng, W. Liu, D. Long, Real-Time Prediction of Docker Container Resource Load Based on a Hybrid Model of ARIMA and Triple Exponential Smoothing 10 (2022) 1386–1401.
- [30] Y. Qiao, C. Li, S. Hao, J. Wu, L. Zhang, Deep or statistical: an empirical study of traffic predictions on multiple time scales, in: Proceedings of the SIGCOMM'22 Poster and Demo Sessions, 2022, pp. 10–12.
- [31] Y.-C. Jin, Q. Cao, Q. Sun, Y. Lin, D.-M. Liu, C.-X. Wang, X.-L. Wang, X.-Y. Wang, et al., Models for COVID-19 data prediction based on improved LSTM-ARIMA algorithms, *IEEE Access* (2023).
- [32] Y. Miao, X. Bai, Y. Cao, Y. Liu, F. Dai, F. Wang, L. Qi, W. Dou, A novel short-term traffic prediction model based on SVD and ARIMA with blockchain in industrial internet of Things, *IEEE Internet of Things Journal* 10 (24) (2023) 21217–21226.
- [33] M.Y. Hassan, Applications of Bigdata Technologies in the Comparison of BMTD and ARIMA Models for the Prediction of Internet Congestion, *IEEE Access* (2024).
- [34] P. Tang, Y. Dong, J. Jin, S. Mao, Fine-grained classification of internet video traffic from QoS perspective using fractal spectrum, *IEEE Transactions on Multimedia* 22 (10) (2019) 2579–2596.
- [35] S. Haytamy, F. Omara, Enhanced qos-based service composition approach in multi-cloud environment, in: 2020 International Conference on Innovative Trends in Communication and Computer Engineering (ITCE), IEEE, 2020, pp. 33–38.
- [36] L. Yang, J. Liu, C. Pan, Q. Zou, D. Wei, MQRP: QoS attribute decision optimization for satellite network routing, in: 2018 IEEE International Conference on Networking, Architecture and Storage (NAS), IEEE, 2018, pp. 1–8.
- [37] F. Hu, M. Zhou, P. Yan, D. Li, W. Lai, K. Bian, R. Dai, Identification of mine water inrush using laser-induced fluorescence spectroscopy combined with one-dimensional convolutional neural network, *RSC advances* 9 (14) (2019) 7673–7679.
- [38] G.E. Hinton, N. Srivastava, A. Krizhevsky, I. Sutskever, R.R. Salakhutdinov, Improving neural networks by preventing co-adaptation of feature detectors, (2012). arXiv preprint arXiv:1207.0580.
- [39] Z.-H. Zhan, J. Zhang, Y. Li, H.S.-H. Chung, Adaptive particle swarm optimization, *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)* 39 (6) (2009) 1362–1381.
- [40] Y.-Y. Zhang, J. Luo, Y. He, S. Ma, K. Liang, Z. Liu, Smart Meter Intrusion Detection Based on APSO-DBN Model, in: 2021 Smart City Challenges & Outcomes for Urban Transformation (SCOUT), IEEE, 2021, pp. 214–217.

- [41] A. Lu, L. Yu, L. Tan, Apso-based optimization algorithm of lstm neural network model, in: 2021 IEEE 5th Advanced Information Technology, Electronic and Automation Control Conference (IAEAC), 5, IEEE, 2021, pp. 2194–2200.
- [42] X. Kan, Y. Fan, Z. Fang, L. Cao, N.N. Xiong, D. Yang, X. Li, A novel IoT network intrusion detection approach based on adaptive particle swarm optimization convolutional neural network, *Information Sciences* 568 (2021) 147–162.
- [43] S. Liu, C. Guo, Retracted: The Simulation Research on Random Disturbance in Multi-Hop Wireless Network, in: 2018 3rd International Conference on Smart City and Systems Engineering (ICSCSE), IEEE, 2018, pp. 655–659.
- [44] Y. Fu, X. Wang, F. Fang, Multi-Objective Multi-Dimensional Resource Allocation for Categorized QoS Provisioning in Beyond 5G and 6G Radio Access Networks, *IEEE Transactions on Communications* (2023).
- [45] Y. Gao, C. Lu, Y. Lian, X. Li, G. Chen, D.B. da Costa, A. Nallanathan, QoS-Aware Resource Allocation of RIS-Aided Multi-User MISO Wireless Communications, *IEEE Transactions on Vehicular Technology* (2023).
- [46] H. Hajizadeh, M. Nabi, K. Goossens, Decentralized configuration of TSCH-based IoT networks for distinctive QoS: A deep reinforcement learning approach, *IEEE Internet of Things Journal* 10 (19) (2023) 16869–16880.
- [47] F. Li, Q. Yuan, T. Pan, X. Wang, J. Cao, MTU-Adaptive In-Band Network-Wide Telemetry, *IEEE/ACM Transactions on Networking* (2024).
- [48] A.C. Wilson, R. Roelofs, M. Stern, N. Srebro, B. Recht, The marginal value of adaptive gradient methods in machine learning, *Advances in neural information processing systems* 30 (2017).
- [49] N.S. Keskar, R. Socher, Improving generalization performance by switching from adam to sg, (2017). *arXiv preprint arXiv:1712.07628*.
- [50] D.H. Wolpert, W.G. Macready, No free lunch theorems for optimization, *IEEE transactions on evolutionary computation* 1 (1) (1997) 67–82.